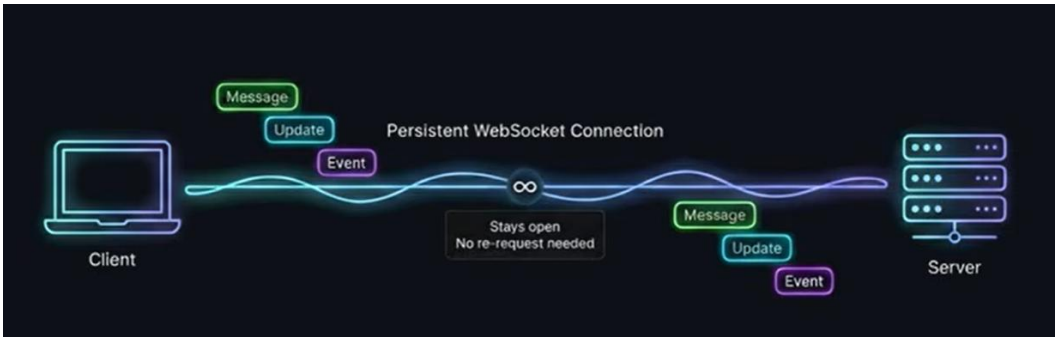
WebSockets

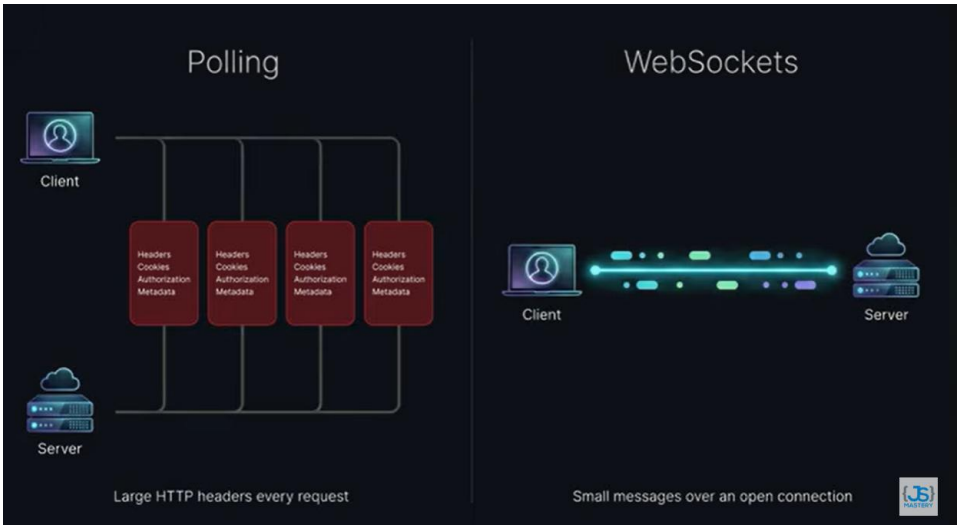
A websocket is a communication protocol that establishes a permanent, two-way (full-duplex) connection between a client and a server. Unlike standard HTTP, which is a request-response protocol where the client must constantly "poll" or ask the server if there is new data, a websocket keeps the connection continuously open. This allows both the server and the client to push messages to each other instantly at any time, making it ideal for real-time applications like chat apps, typing indicators, live sports scoreboards, and multiplayer games



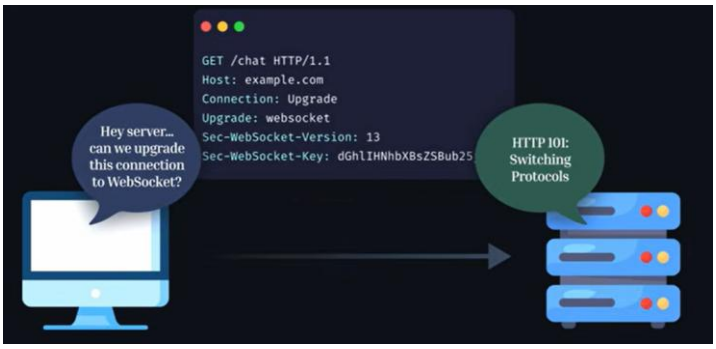
Polling - communication method used by early real-time applications where a client (like a web browser) constantly checks the server every few seconds to ask if there is any new data or if anything has changed

While it technically gets the job done, polling is **highly inefficient and wasteful**

- constant repeated checks wake up the phone over and over again, draining its battery
- send a full HTTP request loaded with large headers



WebSocket Handshake



starts as a normal HTTP request. The browser sends this request with a special header asking the server to "upgrade" the connection. If the server accepts, it replies with an "HTTP 101 switching protocols" response. At that exact moment, the HTTP protocol ends, and a persistent, real-time websocket tunnel is left open

While standard HTTP is "stateless" (meaning the server forgets you after every request), websockets are **stateful**. The server actively holds a reference to your specific socket connection in its memory, which is how it knows exactly where to push instant updates

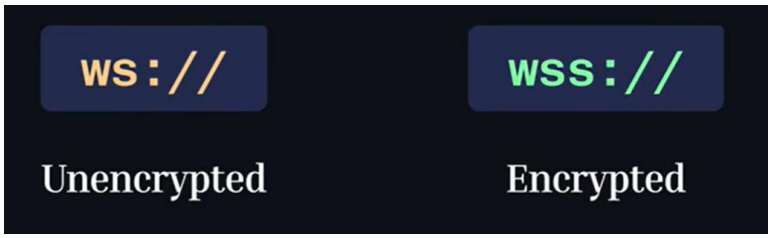
This statefulness also creates a new problem!

A **ghost connection** happens when a client connects to a websocket server, but later loses their connection—such as their device dying, losing Wi-Fi, or closing a laptop—without properly shutting down the socket.

Without a proper disconnect signal, the server assumes the connection is still alive and continues to hold onto it. If too many of these ghost connections accumulate over time, they will silently drain the server's memory, eventually causing it to crash.

*To prevent this, production websocket servers implement a **ping-pong heartbeat** system. The server will periodically send a tiny "ping" impulse through the tunnel. If the client fails to respond with a "pong" within a specific timeframe, the server realizes the client is dead and safely terminates the socket, cleaning up the ghost connection before it can cause memory issues*

Two modes of Websockets:



Data Transfer in WebSockets

1. Data Formats: Text vs. Binary

When pushing data through a websocket, you must choose between two primary formats:

• **Text / JSON:** This is the standard for 99% of applications. It is human-readable, easy to debug, and highly flexible. You typically stringify your data into JSON format to send things like chat messages, live score updates, or typing indicators.

• **Binary:** This format sends raw bytes (using ArrayBuffers or Blobs) instead of text. It takes up less bandwidth and is strictly used when things get "serious"—such as streaming audio or video frames, updating high-speed multiplayer game states, or handling massive payloads.

Under the hood, the server knows exactly what type of data is arriving because the websocket protocol attaches a hidden label called an **"op code"** to every frame of data. This op code identifies whether the incoming frame is text, binary, a ping/pong heartbeat, or a closing command.

2. Message Structure: The Envelope Pattern

A common mistake when transferring data is sending raw, unstructured strings (e.g., `socket.send("hello")`). This sends a "blind packet" leaving the server with absolutely zero context about what the message means.

Instead, real-world applications use the **Envelope Pattern**. Just like mailing a physical package, you don't just send the item; you put it in an envelope and label it. A clean websocket message envelope typically includes:

- **Type:** Tells the server exactly what logic to run (e.g., `chat_message`, `typing_status`, or `join_room`).
- **ID:** A unique identifier used to track the specific message.
- **Payload:** The actual core data you want to transfer.
- **Metadata:** Optional extra context

When the server receives an envelope, it acts as a "switchboard," instantly routing the action based solely on the type field

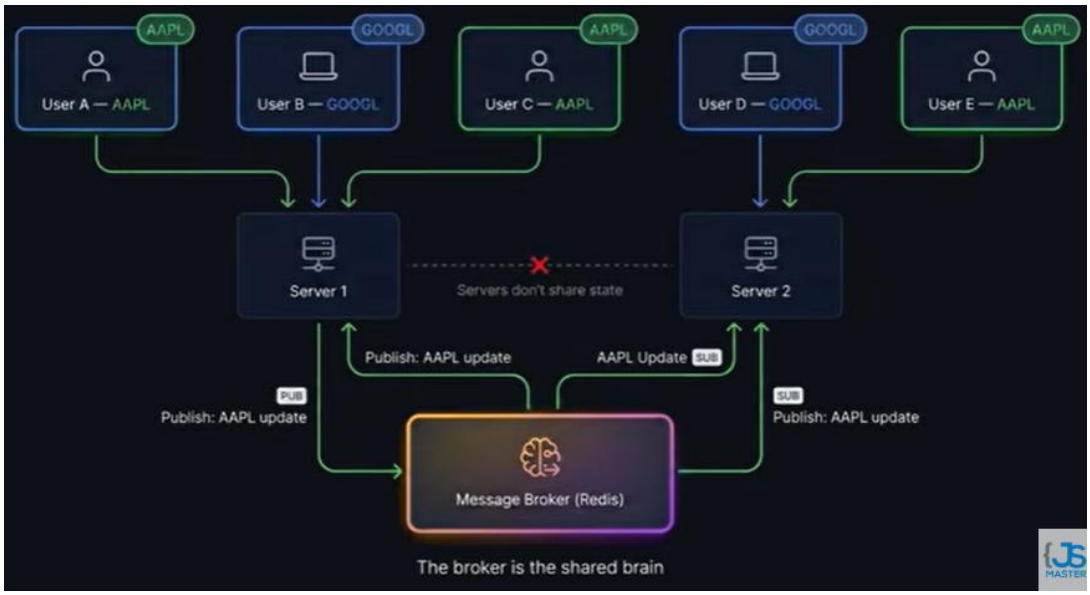
- **Type-based Commands:** Best for applications with a known, fixed list of actions (e.g., chat apps, dashboards, collaborative tools).
- **Topic-based (Pub/Sub):** Best for dynamic channels where users subscribe to specific topics (e.g., stock tickers, live sports engines)

Publish-Subscribe (Pub/Sub) Pattern

The Pub/Sub pattern is an advanced routing architecture that solves the scalability issue of broadcasting every update to every user. In a high-traffic app, you only want to send data to users who actively care about it

- **Subscription:** When a user wants updates, their client sends a subscribe command to the server with a specific topic or ID.
- **Registry:** The server adds the user's socket to a specific "room" or list (often using a Map or Set to prevent duplicate subscriptions).
- **Targeted Broadcast:** When an event happens, the server looks up the specific list of subscribed sockets and only broadcasts the update to that targeted group

Scaling the Pub/Sub pattern across multiple servers requires a central **message broker** (like Redis). Because each server only knows its own connected clients, the broker acts as a central broadcast layer, ensuring an update published on Server 1 reaches subscribed users connected to Server 2



Acknowledgements (ACKs)

Websockets are inherently "fire and forget" and do not have an automatic "200 OK" receipt like HTTP. To build reliability, engineers use the Acknowledgements pattern:

1. The client sends a message with a unique ID.
2. The server processes the action and sends a receipt back containing that exact same ID.
3. If the client does not receive the receipt within a specific timeframe, it knows the message was lost and retries sending it

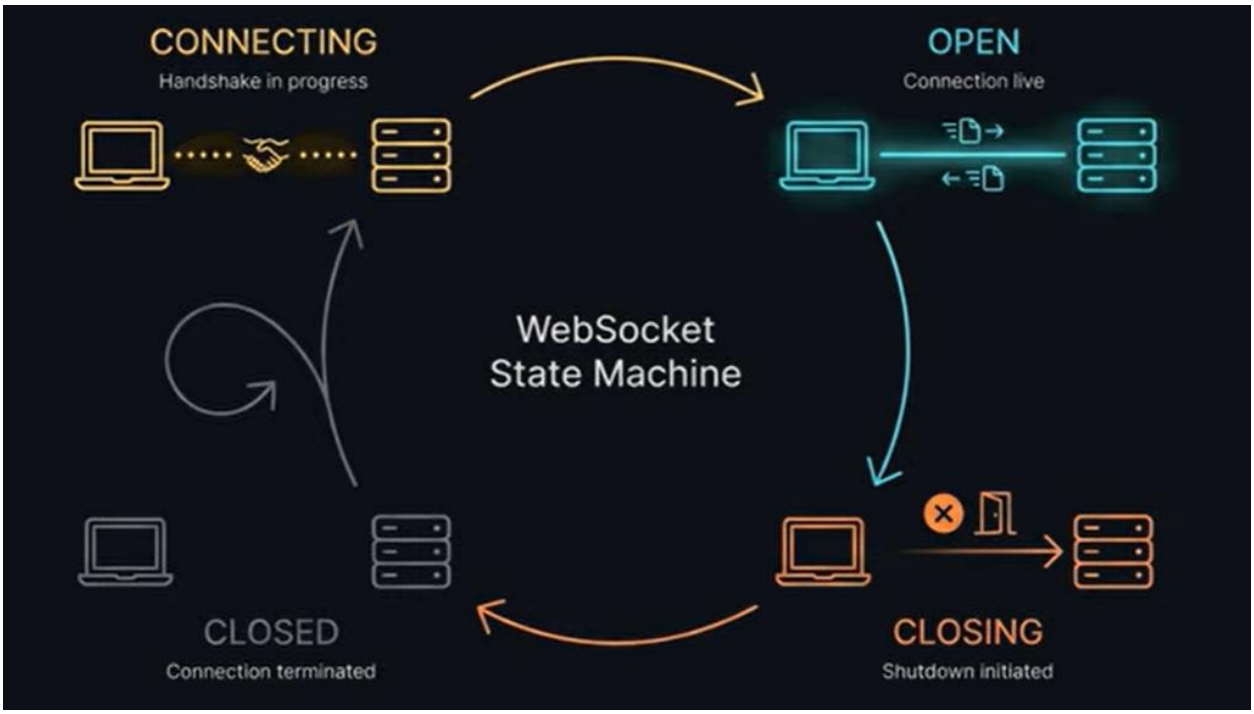
3. Message Routing Patterns

Once the server receives structured data, it needs to decide who gets it. Data transfer routing is usually handled in one of three ways:

- Broadcast (One-to-All):
- Unicast (One-to-One)
- Multicast (One-to-Many)

Websocket LifeCycle

A websocket connection goes through distinct stages during its life cycle. Managing these stages correctly is critical; if you attempt to send data while the connection is in the wrong state, you will encounter broken messages, memory leaks, or application crashes



1. Connecting: The initial HTTP handshake is still happening. You cannot send data yet

2. Open: The "safe zone." The tunnel is live, and this is the only state where it is safe to use the `send` command

3. Closing: The connection is in the process of shutting down. You must stop pushing data

4. Closed: The connection is entirely dead. Once closed, a socket can never be reopened; you must create a brand new websocket instance to reconnect

Websocket management in Server-side

On the server side, your application acts as the host, responsible for tracking multiple clients joining, sending data, crashing, or disconnecting. Using the native ws library in Node.js, you must manage four primary events:

1. Connection: This is your entry point. It fires after the HTTP 101 handshake upgrade is verified. This event gives you access to the specific client's socket and the original HTTP request, allowing you to extract details like their physical IP address or cookies

```
wss.on('connection', (socket, request) => {
  console.log('Client joined:', request.socket.remoteAddress)
})
```

2. Message: Triggers when incoming data arrives. Because the data initially arrives as a raw binary buffer, you must convert it to a string (using .toString()) before your code can read or parse it as JSON

```
socket.on("message",(raw) => {
  const text = raw.toString();
  console.log("Incoming:", text);
});
```

3. Error: This event is critical for stability. Because sockets are fragile (e.g., a user's Wi-Fi flickers), they can throw unexpected errors. If you do not catch these errors, a single bad connection can crash your entire Node process.

```
socket.on("error",(err) => {
  console.log("Socket error:", err.message);
});
```

4. Close: Fires when a user disconnects, and is primarily used for cleaning up memory and removing the client from active broadcast lists

Websocket Management in Client-side

On the client side (typically in a web browser), your application acts as a participant waiting for the server to push events into your code. You initiate the connection by calling new WebSocket() and passing your server's URL. The client listens for these key events:

1. Open: Fires exactly once when the HTTP upgrade succeeds and the tunnel is established. This is the first safe moment you can send data to the server.

```
socket.addEventListener("open",() => {
  console.log("Connected ✅");
  socket.send("Hello server");
});
```

2. Message: Fires every time the server decides to push data down to the client. The client does not need to ask for this data; it arrives automatically, typically as a JSON string, Blob, or ArrayBuffer.

```
socket.addEventListener("message",(event) => {
  const data =JSON.parse(event.data);
  console.log("New data:", data);
});
```

3. Close: Fires when the server intentionally collapses the tunnel, or if the user's network drops. Once a socket reaches the closed state, it is entirely dead and can never be reopened

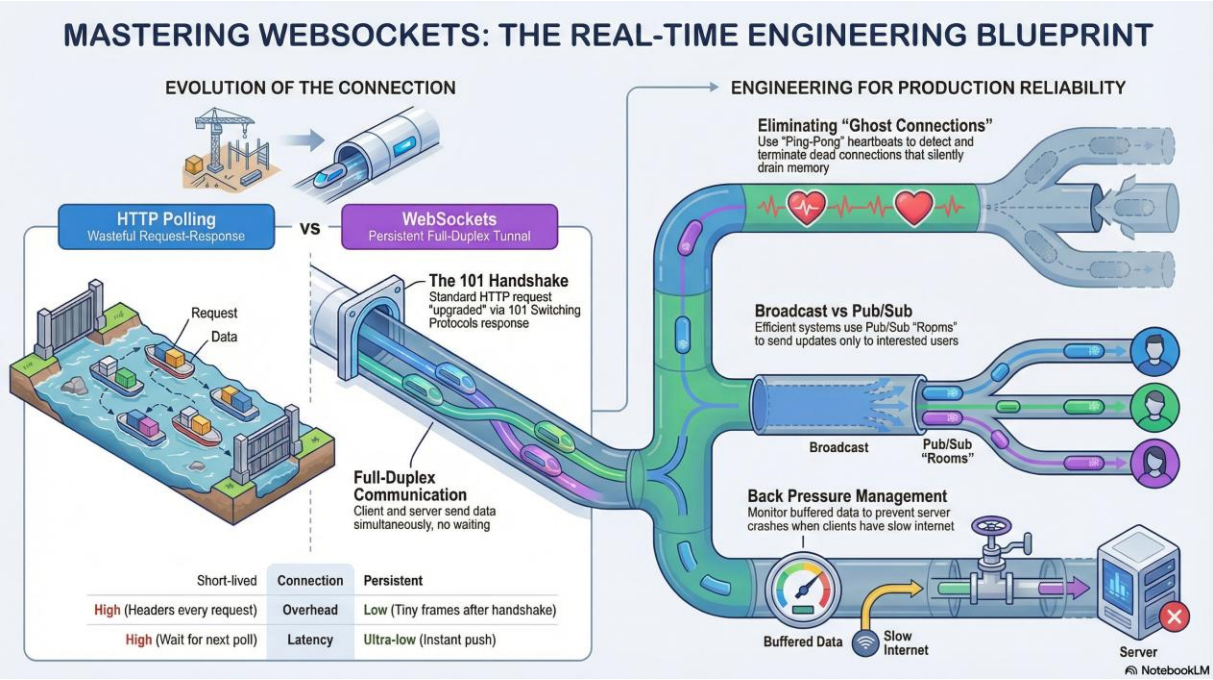
```
socket.addEventListener("close",() => {
  console.log("Disconnected");
});
```

Cheat Sheet

```
// SERVER (ws)
wss.on("connection",(ws) => {
  ws.on("message",(data) => {});
  ws.on("error",(err) => {});
  ws.on("close",() => {});
});

// CLIENT (Browser)
const socket =newWebSocket("ws://...");
socket.onopen =() => {};
socket.onmessage =(e) => {};
socket.onerror =() => {};
socket.onclose =() => {};
```

InfoGraph



Usage of WebSockets

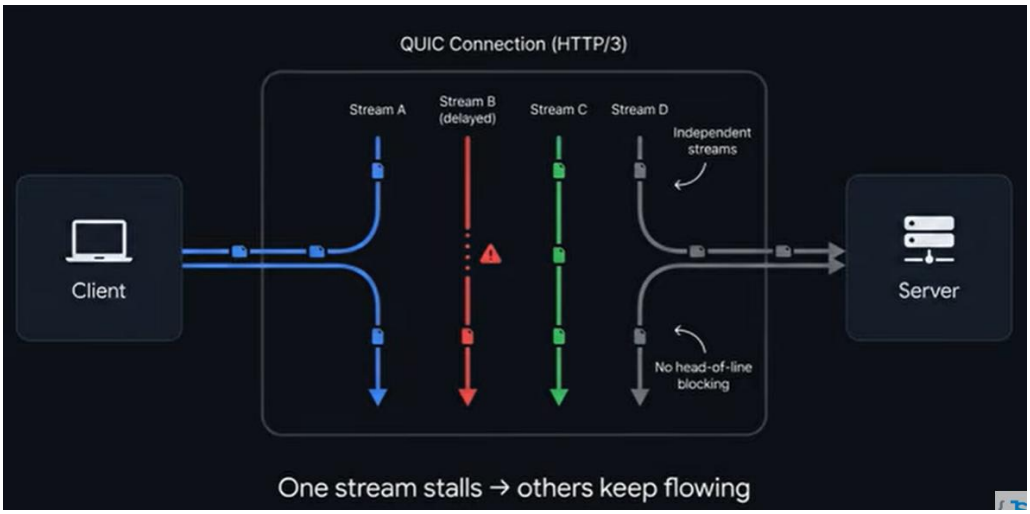
While websockets are highly capable for real-time applications, they are not suitable for every type of data transfer.

1. WebSockets (Two-Way Interactive Apps) WebSockets provide a persistent, full-duplex tunnel where both the client and the server can send messages to each other at any time.

- **Best for:** Chat applications, collaborative tools, multiplayer UI movement, and live dashboards
- **Drawback:** WebSockets are built on TCP. If a single data packet gets lost, the entire stream pauses until that packet is resent—a problem known as "head-of-line blocking"

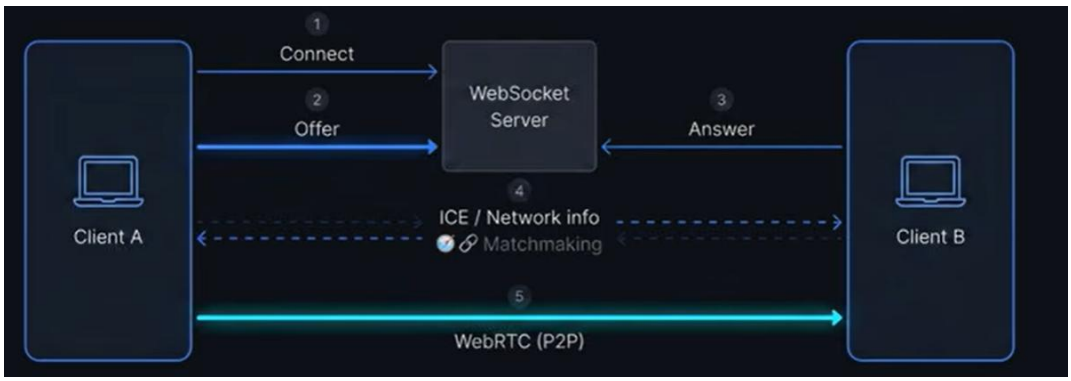
2. WebTransport (Ultra-Low Latency Streaming) WebTransport is an advanced alternative to WebSockets used when peer-to-peer isn't an option but you still need absolute minimal latency. It solves the TCP "head-of-line blocking" issue by using HTTP/3 powered by QUIC.

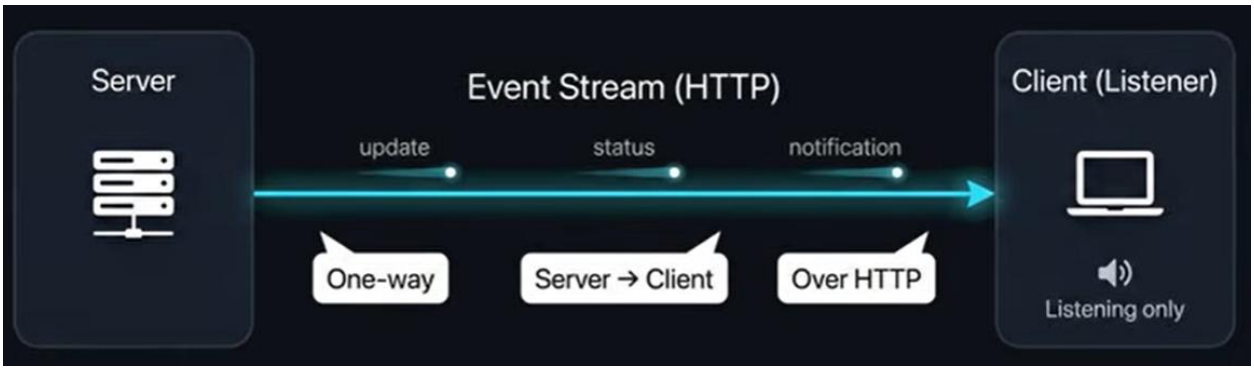
- **Best for:** High-frames-per-second cloud gaming, live media pipelines, and highly competitive real-time systems.
- **Advantage:** It supports multiple independent streams over a single connection; if one stream gets delayed or drops a packet, the other streams keep moving without freezing the whole application



3. WebRTC (Heavy Media & Peer-to-Peer) WebRTC is designed for heavy media routing. Instead of sending data through a central server, WebRTC allows two browsers to connect directly to each other (peer-to-peer), resulting in ultra-low latency.

- **Best for:** Voice calls, video calls (like Zoom), and peer-to-peer file transfers.
- **How it relates to WebSockets:** WebRTC cannot initiate connections on its own; it requires a "signaling" phase to find the other user. Applications typically use WebSockets to exchange the initial network details (the handshake), and then WebRTC takes over to stream the media directly between peers



4. SSE / Server-Sent Events (One-Way Data Streams) SSE is essentially a one-way HTTP stream where the server pushes updates and the client simply listens. There is no full-duplex tunnel, making it much more lightweight than a websocket.

- **Best for:** Stock tickers, news feeds, and live streaming data.
- **Advantage:** It is highly simple, operates over standard HTTP, and handles reconnections automatically. If your user never needs to talk back to the server, websockets are overkill and SSE is the cleaner tool